**Case Study:** MATLAB analysis on Multi Spring Mass Static Equilibrium

**Objective:** Form stiffness matrix for 5 to 10 mass spring chain and compute displacement under applied loads using linear system solvers. Plot the displacement v/s time graph for each mass.

## Comprehensive Description of Multi Spring–Mass System with Damping and Friction

### 1. System Description

The code models a 1-D multi-degree-of-freedom spring–mass system.

- There are N masses placed in a straight line.
- Each adjacent mass is connected by a linear spring.
- The first mass is connected to the left wall using a spring of stiffness $k_0$.
- The last mass is connected to the right wall using a spring of stiffness $k_N$.
- A force is applied to the first mass.
- The masses move horizontally on a surface.
- Friction exists between the masses and the floor.
- Viscous damping is present in the system.

This creates a realistic mechanical model similar to connected vibration absorbers, suspension chains, or distributed mechanical systems.

### 2. Assumptions Taken

To simplify and model the system properly, the following assumptions are made:

1. Motion is **only in one direction (horizontal)**.
2. Springs are **linear** and follow Hooke's Law.
3. Damping is **viscous**, proportional to velocity.
4. Friction is **Coulomb friction**, proportional to μmg.
5. Masses are treated as **point masses**.
6. The surface is flat and gravity is constant.
7. No rotational motion exists.
8. System response is solved numerically using time stepping.

These assumptions make the system mathematically manageable while still capturing real engineering behaviour.

### 3. Mathematical Model of the System

The dynamic equation governing the system is:

$$M\ddot{x}+C\dot{x}+Kx+F_{fric}=F$$

Where:

- (M) = Mass matrix (diagonal)
- (C) = Damping matrix

- (K) = Stiffness matrix
- (x) = Displacement vector
- $\dot{x}$ = Velocity vector
- $\ddot{x}$ = Acceleration vector
- $F_{fric}$= Friction force vector
- F = External force vector

**Mass Matrix (M)**

$$M=diag(m1,m2,...,mN)$$

Each diagonal element represents the inertia of a mass. Since masses do not directly affect each other through inertia, there are no off-diagonal terms.

**Damping Matrix (C)**

$$C=c*I$$

Assumes equal viscous damping for each mass.

**Stiffness Matrix (K)**

The stiffness matrix is assembled based on spring connections.
It follows a **tridiagonal symmetric pattern**:

- For internal masses:

$$K(i,i)=ki-1+ki$$

- Off-diagonal terms:

$$K(i, i-1) = -k_{i-1}$$
$$K(i, i+1) = -k_i$$

For boundary masses:

First mass:

$$K(1,1)=k_0+k_1$$

Last mass:

$$K(N,N)=kN-1+kN$$

**Pattern of Stiffness Matrix**

It has this structure:

$$\begin{bmatrix} k_0+k_1 & -k_1 & 0 & \dots \\ -k_1 & k_1+k_2 & -k_2 & \dots \\ 0 & -k_2 & k_2+k_3 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

**Key Characteristics:**

- Symmetric
- Sparse
- Tridiagonal

- Diagonally dominant
- Physically represents force balance at each mass

This structure is typical in vibration and finite element problems.

### 4. Friction Model:

Friction is modelled as:

$$F_{fric} = \mu mg \cdot \text{sign}(v)$$

- Opposes direction of motion
- Depends on velocity sign
- Acts independently on each mass
- Makes system nonlinear

### 5. Time Integration Method:

The system is solved using **explicit time stepping** (Euler method).
Acceleration is calculated using:

$$a = M^{-1}(F - Cv - Kx - F_{fric})$$

Then:

$$v_{new} = v + dt \cdot a$$

$$x_{new} = x + dt \cdot v$$

This updates displacement and velocity step by step.

### 6. How Loops and Conditionals Work

For loops are used to:

1. Collect mass values from the user.
2. Collect stiffness values of springs.
3. Construct the stiffness matrix row by row depending on position of each mass.
4. Run the simulation across many time steps.

The time loop continues until steady state is reached or the maximum number of steps is completed.

If conditions are used in two main areas:

1. During stiffness matrix construction to treat the first and last masses differently from internal masses.
2. During simulation to check whether the system has reached steady state.

The steady state condition verifies whether:

- The maximum velocity is below a defined tolerance.

- The maximum acceleration is below a defined tolerance.

If both values are sufficiently small, the system is considered stabilized and the loop stops.

## 7. Time Stamp and Displacement vs Time

### Time Tracking

$$t(step+1) = t(step) + dt;$$

This increases time by a fixed increment $dt$ at each iteration.
At the same time:

- Displacement values are stored for each mass.
- Velocity values are stored.
- Acceleration values are stored.

After simulation completes, the stored time array and displacement matrix are used to generate the displacement versus time graph.

This graph clearly shows how each mass moves from its initial condition until equilibrium is reached.

### 8. Engineering Interpretation of Results

The graph shows:

1. Initial acceleration due to applied force.
2. Oscillatory behaviour due to springs.
3. Energy loss due to damping.
4. Motion resistance due to friction.
5. Gradual stabilization to static equilibrium.

Final displacement:

$$Kx_{final}=F-F_{fric}$$

Thus, the system settles into a balance of:

- Spring forces
- Friction forces
- Applied force

## 9. Engineering Importance:

This model represents:

- Multi-body vibration systems
- Structural dynamic chains
- Vehicle suspension modelling
- Conveyor or linked mechanical assemblies
- Finite element discretized structures

It combines:

- Mass inertia
- Elastic restoring force

- Energy dissipation (damping)
- Nonlinear friction
- External loading

The program provides a complete numerical simulation of real mechanical behaviour in a controlled and scalable way.

## 10. Conclusion:

This MATLAB code successfully models a realistic multi-degree-of-freedom spring–mass system with:

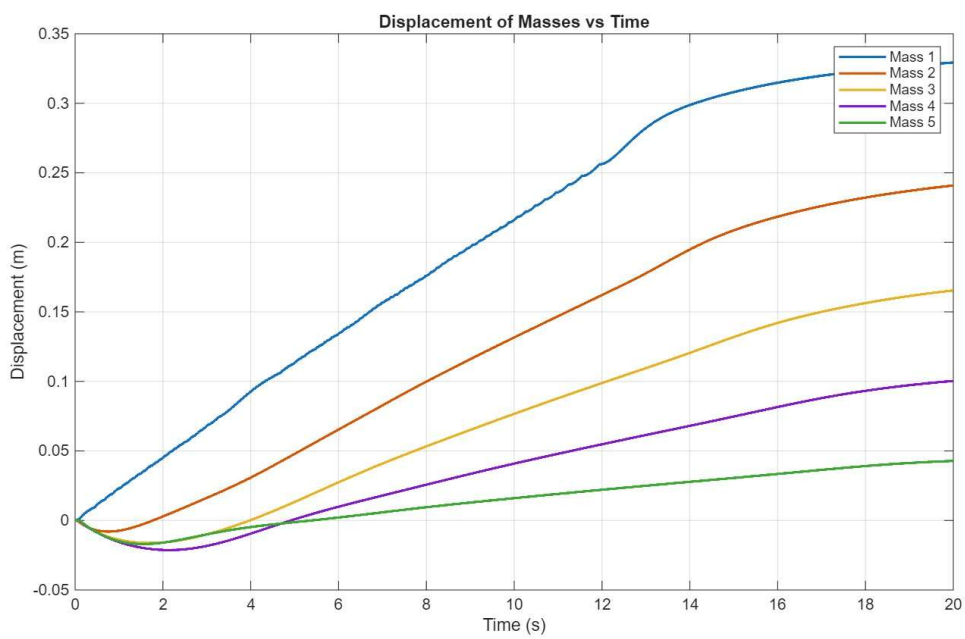
- Boundary springs
- Inter-mass springs
- Viscous damping
- Coulomb friction
- External force
- Time-domain numerical solution

- The stiffness matrix follows a structured tridiagonal form typical of connected mechanical systems.
- The time-stepping loop simulates real-time motion evolution.
- The stopping condition ensures detection of steady state.

Overall, the code is a complete engineering simulation tool for studying dynamic behaviour of interconnected mechanical systems under damping and friction effects.

**Result:**

Command Window

```
Enter number of masses: 5
Enter stiffness of spring at left wall (N/m): 100
Enter stiffness of spring 1 (between mass 1 and 2) (N/m): 150
Enter stiffness of spring 2 (between mass 2 and 3) (N/m): 200
Enter stiffness of spring 3 (between mass 3 and 4) (N/m): 250
Enter stiffness of spring 4 (between mass 4 and 5) (N/m): 300
Enter stiffness of spring at right wall (N/m): 350
Enter load applied on first mass (N): 1000
Displacement of each mass (m):
    6.8610
    4.7683
    3.1988
    1.9432
    0.8969
```

**Graph:**